Computers, programs, and Java

COMP 110/L Chapter 1

Introduction to Algorithms and Programming

Bahram Zartoshty

Learning objectives

Explain hardware, software, and the operating system.

Describe an algorithm and distinguish syntax from meaning.

Use the JDK and an editor or IDE to compile and run Java.

Read console output and diagnose common programming errors.

Computer science and problem solving

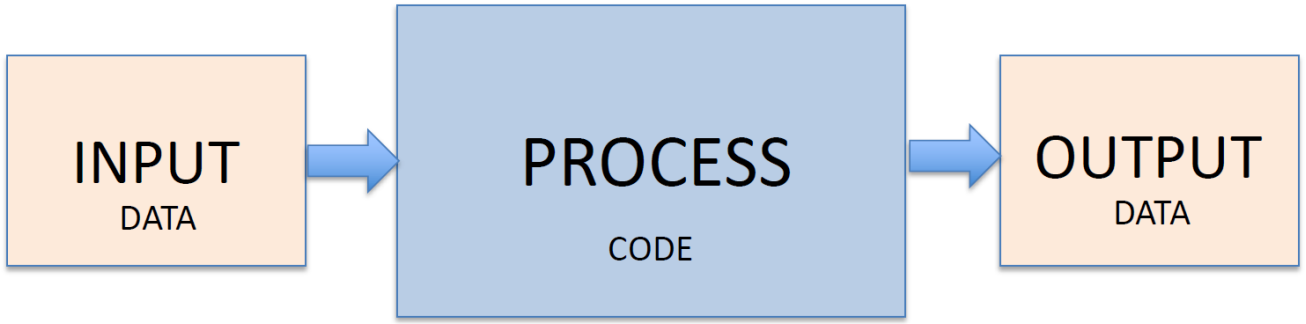
A problem statement describes the result we want.

An algorithm describes precise steps to produce that result.

A program expresses those steps in a programming language.

A computer executes instructions. People evaluate whether they solve the problem.

The input-process-output model



Input: data supplied to the program.

Processing applies an algorithm. Output communicates the result.

Discussion 1: a vending machine

A vending machine accepts a product selection and payment.

What are its inputs, processing steps, and outputs?

How should it respond to insufficient payment or an unavailable item?

Discuss with a partner for 2 minutes. Give one normal case and one edge case.

Computer hardware

Component	Role
CPU	Executes machine instructions
Main memory (RAM)	Holds data and instructions in use
Storage: SSD or hard disk	Retains files when power is off
Input and output devices	Receive data and present results
Communication devices	Exchange data with other systems

Hardware refers to the physical components of a computer system.

Programs and software

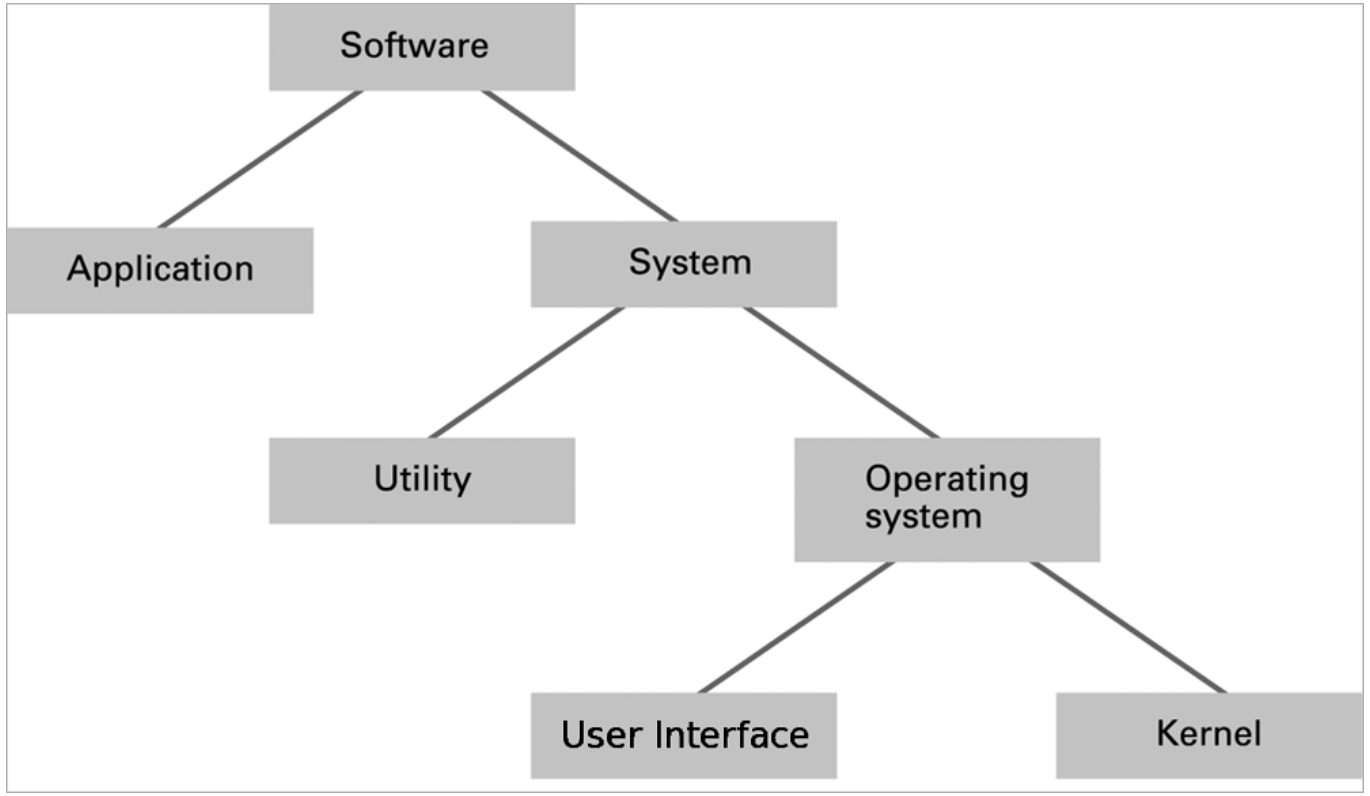
A computer program is a sequence of instructions.

Software includes programs and related data that perform tasks.

Java is a high-level language with support for object-oriented programming.

A programming language provides rules for expressing instructions.

Software categories



Application software serves user tasks.

System software supports the computer and other programs.

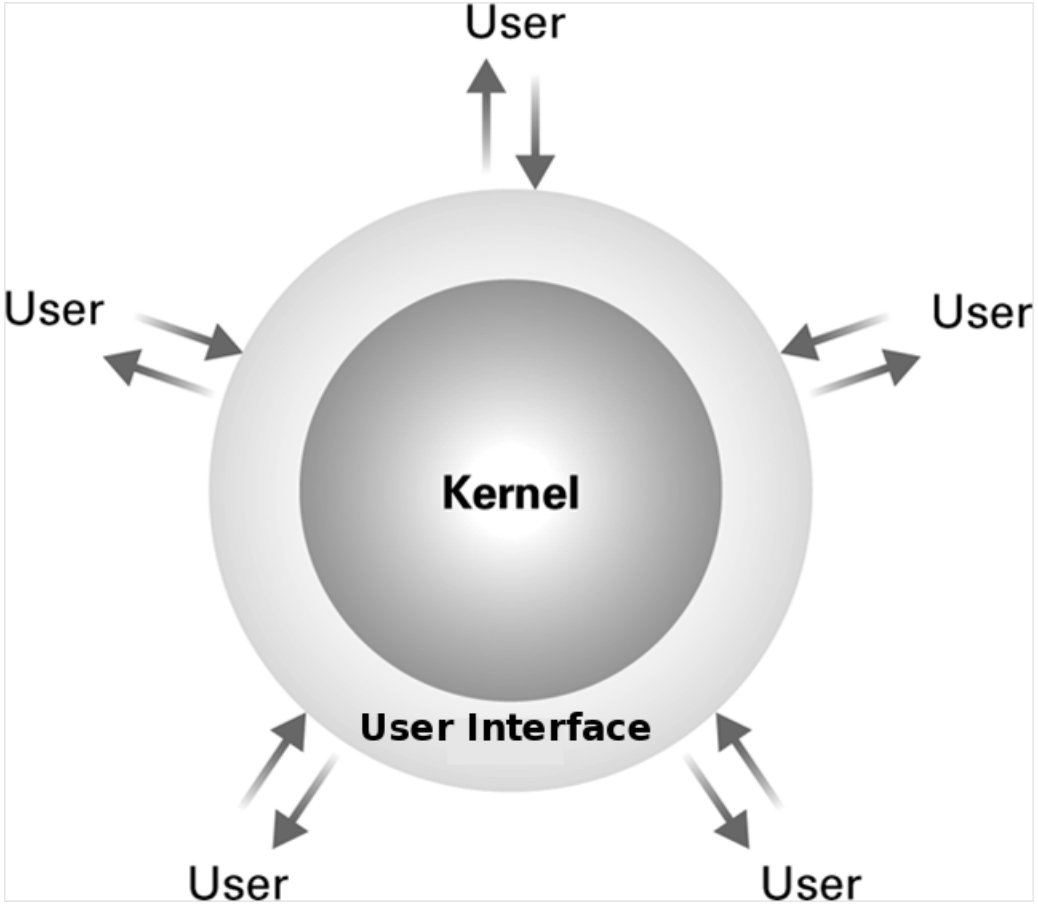
Utilities provide support functions such as backup or file maintenance.

Operating system responsibilities

Responsibility	Example
Files	Organize and access stored data
Memory	Allocate and protect working memory
Devices	Coordinate input and output through drivers
Scheduling	Choose which tasks run on the CPU
User interaction	Offer a graphical interface or command shell

The kernel coordinates core services. User interfaces expose ways to request work.

User interfaces and the kernel



A shell accepts text commands.

A GUI uses visual controls such as windows and menus.

Both rely on operating-system services to perform work.

Quiz 1: hardware and software

Which statement is correct?

A. RAM permanently retains every file after power is removed.

B. The operating system manages resources used by programs.

C. A keyboard is application software.

D. A Java source file is a CPU.

Computational thinking

Break a large problem into smaller parts.

Identify the information that matters and ignore irrelevant details.

Express a solution as clear steps.

Test those steps with ordinary inputs and edge cases.

An algorithm for circle area

Step	Action
1	Obtain a radius r
2	Check that r is nonnegative
3	Compute area = pi × r × r
4	Display the area with square units

The algorithm states the solution before choosing Java syntax.

Discussion 2: precise instructions

A program must calculate the average of two test scores.

Write the steps without using Java code.

What information must the problem statement specify?

How would you test your proposed algorithm?

Syntax and semantics

Concept	Meaning	Example
Syntax	Rules for forming valid statements	A Java statement needs the required punctuation
Semantics	Meaning or behavior of a valid statement	A formula determines the value assigned

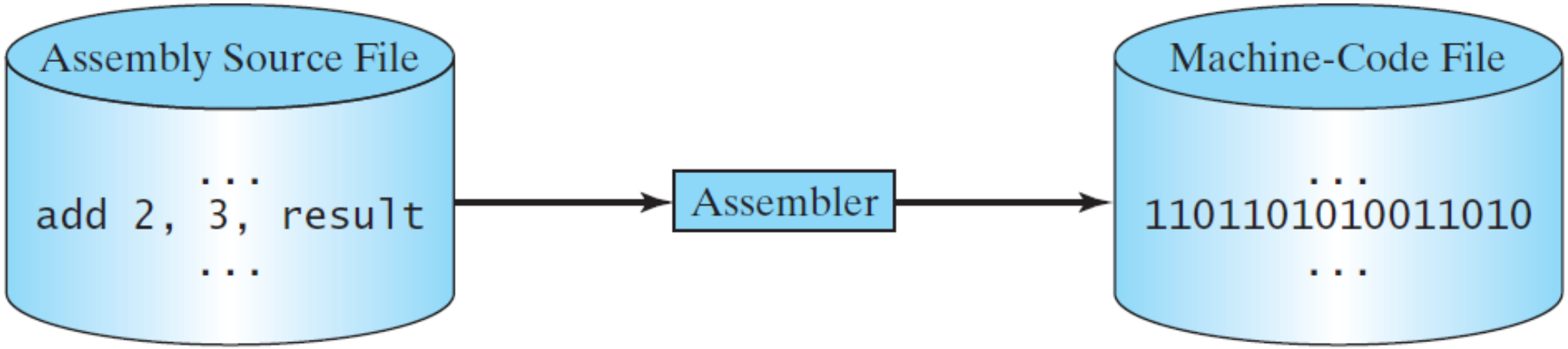
Grammar asks whether a statement is well formed. Meaning asks what it does.

Levels of programming languages

Level	How instructions appear	Translation
Machine language	Encoded instructions for a CPU architecture	Executes on a matching processor
Assembly language	Mnemonics and architecture-specific operands	An assembler produces machine code
High-level language	Statements such as area = r * r * Math.PI;	A compiler or interpreter processes the source

The language level describes how programmers express operations.

Assembly source and an assembler



An assembler translates mnemonics into encoded machine instructions.

The exact instruction names and operands depend on the CPU architecture.

A Java assignment statement

```
double radius = 5.0;
double area = radius * radius * Math.PI;
```

The left side names the variable that receives the value.

The right side computes a value. The statement ends with a semicolon.

Quiz 2: assignment syntax

Assume area and radius are already declared double variables.

Which statement correctly assigns the circle area?

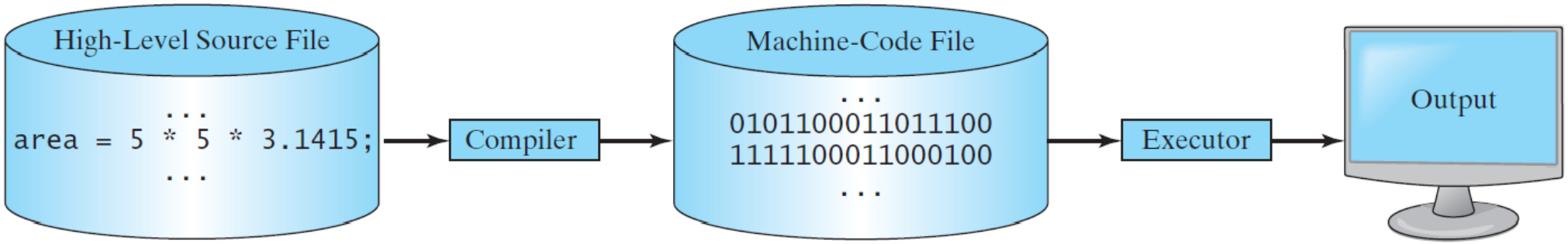
A. radius * radius * Math.PI = area;

B. area = radius * radius * Math.PI;

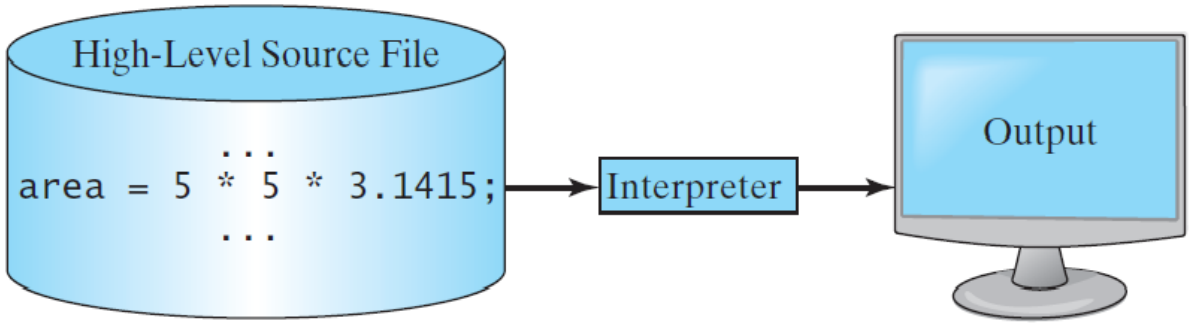
C. area = radius * radius * Math.PI

Compilation and interpretation

Compilation produces translated code for later execution



Interpretation performs operations through a language runtime



The Java development workflow

Stage	What happens
Edit	Write and save Welcome.java
Compile	javac checks source and produces Welcome.class
Load and verify	The JVM loads classes and checks bytecode structure
Execute	The runtime executes the program
Evaluate	Compare the result with the requirement and revise

A successful compilation does not guarantee a correct result.

Java terminology

Term	Role
Language specification	Defines Java syntax and semantics
API	Documents available classes and methods
JDK	Provides development tools such as javac
JVM	Executes Java bytecode
IDE	Combines an editor with development tools

An IDE uses a configured JDK. Installing an IDE and selecting its JDK are separate steps.

Java libraries and APIs

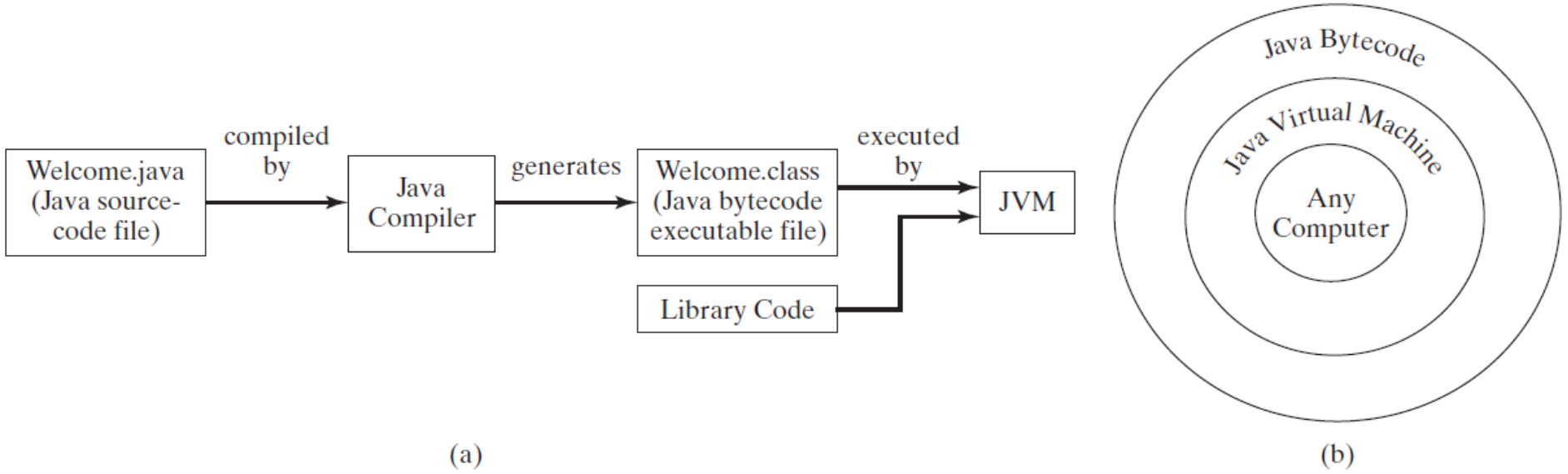
```
System.out.println("Welcome to Java!");

System.out.println(Math.PI);
```

System.out provides console output. println prints a value and ends the line.

Math.PI supplies a value already defined by the standard library.

Bytecode and portability



A compatible JVM executes compiled Java bytecode on its host system.

The JVM implementation depends on the operating system and processor.

Discussion 3: moving a Java program

A class file runs on one computer but fails on another.

What would you check before changing the algorithm?

How does the JVM help portability? What does it still depend on?

Discuss for 2 minutes and propose a short checklist.

Quiz 3: Java tools

Match each term to its role: JDK, JVM, API, IDE.

1. Executes bytecode.

2. Describes available library operations.

3. Integrates editing, building, and debugging.

4. Supplies development tools such as the compiler.

A first Java application

```
// File: Welcome.java
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Save the source as Welcome.java. Java names are case sensitive.

This chapter uses the conventional public static void main entry point.

Anatomy of the first program

Element	Purpose
public class Welcome	Declares the application class
main(String[] args)	Conventional starting method
{ and }	Delimit class and method bodies
System.out.println(...)	Prints to the standard output stream
"Welcome to Java!"	A string literal
;	Terminates this statement

Keywords, identifiers, punctuation, and literals play different roles.

Console output: print and println

```
System.out.print("Welcome ");
System.out.println("to Java!");
System.out.println("Chapter 1");
```

print leaves the cursor on the current line.

println ends the current line after printing its value.

Quiz 4: predict the output

```
System.out.print("COMP ");
System.out.println("110");
System.out.println("Java");
```

Write the exact output. Preserve spaces and line breaks.

Assume these statements execute in order inside main.

Compile and run from a terminal

```
javac -version
java -version

javac Welcome.java
java Welcome
```

Run these commands from the folder containing Welcome.java.

The class-launch command uses Welcome without the .class extension.

Common setup and launch problems

Symptom	First check
javac is not recognized	JDK installation and PATH configuration
File not found	Current folder and source filename
Public class / filename mismatch	Exact spelling and capitalization
Could not find or load main class	Class name, classpath, and compilation output
Old output after editing	Save the source and compile it again

Use the first meaningful diagnostic to decide the next check.

The same workflow in an IDE

Create a Java project and select the configured JDK.

Create Welcome.java and enter the application code.

Save, build, and run the application. Read the output window.

Use compiler diagnostics and the debugger to investigate problems.

Quiz 5: compilation and execution

Welcome.java declares public class Welcome. You use the two-step workflow.

1. Which command compiles this file?

2. Which command launches the compiled class?

3. You edit and save a message. What must happen before the new output appears?

Programming style and documentation

Practice	Reason
Meaningful names	Communicate the purpose of data and classes
Consistent indentation	Show which statements belong to each block
Spaces and blank lines	Separate expressions and related sections
Useful comments	Explain intent, assumptions, or non-obvious reasoning
Consistent brace style	Make the structure easier to scan

Style supports human understanding. The compiler still enforces syntax.

Discussion 4: comments that help

Compare these comments above the same statement: area = radius * radius * Math.PI;

A. Multiply radius by radius and Math.PI.

B. Compute the circle area in square centimeters from a radius in centimeters.

Which comment helps a future reader more? What assumption should be checked?

Three kinds of programming problems

Category	When it appears	Example
Compile-time error	Source fails a language rule	A required quote or semicolon is missing
Runtime failure	Execution encounters a problem	Integer division by zero
Logic error	Result violates the intended requirement	Using pi × r instead of pi × r × r

A running program can still be wrong. A handled exception need not terminate the application.

Quiz 6: repair the syntax

```
public class Welcome {
  public static main(String[] args) {
    System.out.println("Welcome to Java!);
  }
}
```

Identify two independent errors. Write the corrected header and statement.

A runtime failure

```
public class DivideDemo {
  public static void main(String[] args) {
    int divisor = 0;
    System.out.println(1 / divisor);
  }
}
```

This source compiles, but integer division by zero throws an exception.

An unhandled ArithmeticException stops this execution.

Quiz 7: a program with the wrong result

```
public class Circle {
  public static void main(String[] args) {
    double radius = 5.0;
    System.out.println(radius * Math.PI);
  }
}
```

Requirement: display the area of a circle with radius 5.

What is wrong? Classify the problem and correct the expression.

A repeatable debugging process

State the expected result before running the program.

Reproduce the problem with a small, specific input.

Read diagnostics or trace the values used by the program.

Change one cause, then rerun the original and related tests.

Discussion 5: testing the circle program

A program prints a plausible number for radius 1.

Is that enough evidence that its circle-area formula is correct?

Choose another radius that would expose a missing multiplication.

Explain why your test is useful before running the program.

Quiz 8: exit ticket

1. What is the difference between an algorithm and a program?

2. Which file does javac normally produce from Welcome.java?

3. A program compiles and runs but prints the wrong area. What kind of problem is it?

4. Why are testing and readable code both useful?

Independent practice

Create Welcome.java using the conventional main method.

Print your name and a short course message on separate lines.

Compile and run it. Then change the message and repeat the workflow.

Record one compiler error you introduced and how you repaired it.